CREATE AN API FOR CREATING AND VIEWING TODO ITEMS **USING FLASK**

AIM: To-Do List REST API using Flask –

Step 1: Install flask

pip install flask

Writing Flask Code

Created a Python file (my_flask_app.py) and wrote Flask code to implement routes for the API. The following routes were created:

- `/` : Home route (returns welcome message)

- `/todos` : GET method to fetch all tasks

- `/todos` : POST method to add new task

- `/todos/<id>` : GET method to fetch task by ID

- `/todos/<id>` : PUT method to update a task

```
from flask import Flask, request, jsonify
import datetime
app = Flask(__name__)
todos = []
next_id = 1
@app.route('/todos', methods=['POST'])
def create_todo():
global next_id
data = request.get_json()
```

```
task = data.get('task')
status = data.get('status')
date = datetime.datetime.now()
if task is None:
return jsonify({'error': ' Task is required'}), 400
new_todo = {'id': next_id, ' task': task, 'date':date, 'status':status}
todos.append(new_todo)
next_id += 1
return "Task added", 201
@app.route('/todos', methods=['GET'])
def get_todos():
return jsonify(todos)
@app.route('/todos/<int:todo_id>', methods=['GET'])
def get_todo(todo_id):
todo = next((todo for todo in todos if todo['id'] == todo_id), None)
if todo is None:
return jsonify({'error': 'Todo not found'}), 404
return jsonify(todo)
@app.route('/todos/<int:todo_id>', methods=['PUT'])
def update_todo(todo_id):
data = request.get_json()
task = data.get('task')
if task is None:
```

```
return jsonify({'error': 'Todo is required'}) , 400
todo = next((todo for todo in todos if todo['id'] == todo_id), None)
if todo is None:
return jsonify({'error': ' todo not found'}), 404
todo['task']=task
return jsonify(todo)
@app.route('/todos/<int:todo_id>', methods=['DELETE'])
def delete_todo(todo_id):
global todos
todos = [todos for todo in todos if todo['id'] != todo_id]
return jsonify("TASK DELETED")
if __name__ == '__main__':
app.run(debug=True)
```

Alternate code

```
from flask import Flask, request, jsonify
import datetime
app = Flask(__name__)
todos = []
next_id = 1
# Home route (for browser display)
@app.route('/')
```

```
def home():
return "<h2> Welcome to the To-Do Flask API!</h2><p>Use
<code>/todos</code> to see tasks or POST new ones.</p>"
# Create a new todo
@app.route('/todos', methods=['POST'])
def create_todo():
global next_id
data = request.get_json()
task = data.get('task')
status = data.get('status', 'pending')
date = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
if task is None:
return jsonify({'error': 'Task is required'}), 400
new_todo = {
'id': next_id,
'task': task,
'status': status,
'date': date
}
todos.append(new_todo)
next_id += 1
return jsonify({'message': 'Task added successfully!', 'todo': new_todo}), 201
# Get all todos
```

```
@app.route('/todos', methods=['GET'])
def get_todos():
return jsonify(todos)
# Get all todos
@app.route('/todos', methods=['GET'])
def get_todos():
return jsonify(todos)
# Get single todo by ID
@app.route('/todos/<int:todo_id>', methods=['GET'])
def get_todo(todo_id):
todo = next((todo for todo in todos if todo['id'] == todo_id), None)
if todo is None:
return jsonify({'error': 'ToDo not found'}), 404
return jsonify(todo)
# Update a todo
@app.route('/todos/<int:todo_id>', methods=['PUT'])
def update_todo(todo_id):
data = request.get_json()
task = data.get('task')
status = data.get('status')
todo = next((todo for todo in todos if todo['id'] == todo_id), None)
if todo is None:
return jsonify({'error': 'ToDo not found'}), 404
```

```
if task:
todo['task'] = task
if status:
todo['status'] = status
return jsonify({'message': 'ToDo updated successfully!', 'todo': todo})
if __name__ == '__main__':
app.run(debug=True)
```

4. Running the Flask Server

Use IDLE or terminal to run the file:

python my_flask_app.py

Access the API in browser at http://127.0.0.1:5000/

5. Testing the API

{ "task": "Complete practical", "status": "pending" }

6. Output

- First visit to `/todos` showed empty list: []

- After POST request, GET `/todos` showed the added tasks in JSON format.

Testing POST Request using Postman

Objective:

Add a new task to the To-Do API using a POST request via **Postman** tool.

Steps:

1. **Open Postman application** on your system.
2. **Set Request Type to POST** from the dropdown.
3. In the URL bar, enter:

http://127.0.0.1:5000/todos

1. Click on the **Body** tab.
2. Select the **raw** option.
3. From the dropdown next to raw, select **JSON** (application/json).
4. Enter the JSON body as:

{ "task": "Do Task",
"status": "pending" }

5. Click the **Send** button.

Output:
Status: 201 CREATED

- Server responded with a message:

{
"message": "Task added successfully!",
"todo": {
"id": 1,
"task": "Do Task",
"status": "pending",
"date": "2025-07-22 08:47:55"
}